Listing 2-1. *RGB Blink Source Code*

```
/* Project 1: RGB Blinky
   Uses a single RGB LED to cycle through three colors.
*/

void setup() {
  pinMode(9, OUTPUT);      // sets digital pins as outputs
  pinMode(10, OUTPUT);
  pinMode(11, OUTPUT);
}

void loop() {
  digitalWrite(9, HIGH);   // turns on red
  digitalWrite(11, LOW);   // turns off blue
  delay(1000);             // waits for 1 second
  digitalWrite(10, HIGH);  // turns on green
  digitalWrite(9, LOW);    // turns off red
  delay(1000);             // waits for 1 second
  digitalWrite(11, HIGH);  // turns on blue
  digitalWrite(10, LOW);   // turns off green
  delay(1000);             // waits for 1 second
}
```

Source Code Summary

Our first project sketch takes the Blink example that we uploaded in the first chapter and builds on it by adding the additional two colors available in our RGB LED, connected to pins 9, 10, and 11 rather than the built-in LED on pin 13, to blink each of the three colors in turn. As I will explain in this chapter, there are two groups of code in this sketch called functions grouped by matching curly braces, `{ }`, each named `setup()` and `loop()`—ignore the keyword `void` for the time being. We use `setup()` to configure the state of input and output pins, as well as other actions that we only need to run once. Conversely, `loop()` will perform whatever actions are inside of it over and over again for an eternity, assuming limitless power as a prerequisite. Each line of code is executed inside a function, in order, from the top to the bottom. Inside the `loop()` function, when the last line of code is reached the function will repeat, beginning again with the first line of the `loop()` function.

Inside of each of these functions are statements that can include calls to other functions, such as `pinMode()`, `digitalWrite()`, and `delay()`. The function `pinMode()` is used to set a specific pin to either `INPUT` or `OUTPUT`. `digitalWrite()` is used to set the state of an output pin to `HIGH` or `LOW`, effectively used here to turn on or off an LED. The last function, `delay()`, does as it's named and delays the program for a specified number of milliseconds. We will spend the rest of our time together looking at these and other functions and statements that we will use when writing code for the Arduino platform, but first let's start with the basic structure of Arduino C.

The Structure of Arduino C

Our first project will provide an interesting backdrop for understanding what's going on with the basic structure of programming an Arduino sketch. This section will attempt to explain some of the larger